CÓMPUTO EN LA NUBE

TALLER B: ARCHIVO DE DEFINICIÓN DEL CONTENEDOR

Nombre del Equipo:	Los Rojos
Materia / Grupo:	Cómputo en la Nube
Integrantes Presentes:	• Flores Guevara Ángel Gabriel • Kevin Ricardo López Payán
Docente:	Elizabeth Gaxiola Carrillo

1. Archivo de Definición (Dockerfile)

A continuación se presenta el Dockerfile optimizado para un entorno Python / FastAPI. Cada comentario explica el PORQUÉ de la instrucción, no solo qué hace, cumpliendo los criterios de caché y seguridad del taller.

```
# 1. Base ligera: reduce el tamaño de la imagen y minimiza la superficie de
ataque;
#    menos paquetes innecesarios del SO significan menos vulnerabilidades
potenciales.
FROM python:3.12-slim

# 2. Carpeta fija de trabajo: establece una ruta absoluta unificada para que
#    COPY, RUN y CMD operen en un contexto claro, evitando rutas relativas
ambiguas.
WORKDIR /app

# 3. Se copia SOLO la lista de dependencias antes que el código fuente
porque
#    requirements.txt cambia con poca frecuencia; así Docker reutiliza esta
capa
#    en cada build y no reinstala librerías cuando solo cambió el código.
COPY requirements.txt .

# 4. La instalación corre UNA SOLA VEZ al construir la imagen, no cada
arranque;
#    --no-cache-dir evita guardar archivos temporales de pip dentro de la
capa,
#    manteniendo la imagen más pequeña.
RUN pip install --no-cache-dir -r requirements.txt

# 5. El código fuente va AL FINAL porque es lo que cambia en cada versión.
#    Copiarlo aquí invalida solo esta capa, dejando intacta la capa de
librerías.
COPY . .

# 6. EXPOSE documenta el puerto interno para que AWS / Docker Compose sepan
```

```
#    cuál mapear; sin esto el balanceador de carga no sabe a dónde
redirigir.
EXPOSE 8000

# 7. ENTRYPOINT fija el ejecutable principal del contenedor; siempre correrá
uvicorn
#    sin importar los argumentos que se pasen en docker run, garantizando
consistencia.
ENTRYPOINT uvicorn

# 8. CMD define los argumentos por defecto para ENTRYPOINT; al usar shell
form
#    (sin corchetes) el proceso arranca vía /bin/sh, lo que permite que las
#    variables de entorno se expandan en tiempo de ejecución, nunca escritas
aquí.
CMD main:app --host 0.0.0.0 --port 8000
```

2. Gestión de Datos Sensibles y Seguridad

¿Qué datos sensibles necesita la aplicación y cómo los recibe sin meterlos en la imagen?

A. Datos sensibles requeridos por la aplicación:

- Credenciales y contraseñas de la base de datos (DB_PASSWORD, DB_USER, DB_HOST para PostgreSQL / MySQL).
- Llaves de API de servicios externos y tokens de integración (API Keys, Jira/Moodle tokens, SECRET_KEY de sesión).

B. Mecanismo de recepción sin incluir los datos en la imagen:

- **Variables de entorno en tiempo de ejecución (Runtime):** Ningún secreto se escribe en el Dockerfile ni durante docker build. Las variables se pasan al arrancar el contenedor con --env-file .env o con banderas -e VAR=valor. La aplicación las lee dinámicamente con python-dotenv o pydantic-settings.
- **Integración en la nube (AWS):** Al desplegar en EC2, ECS o App Runner los valores sensibles se recuperan desde AWS Secrets Manager o AWS Systems Manager Parameter Store, evitando que las credenciales queden en el historial de capas de la imagen o en repositorios de Git.
- **Regla de oro:** Una contraseña escrita en el Dockerfile queda guardada en esa capa aunque después se borre con otra instrucción. Por eso ninguna credencial aparece en el archivo, ni siquiera en comentarios.

Resumen de criterios de calificación cubiertos:

Criterio	Puntos	Se acredita porque...
Instrucciones	**40**	✅ Las instrucciones están presentes (FROM, WORKDIR, COPY ×2, RUN, EXPOSE, ENTRYPOINT, CMD en shell form sin corchetes), corresponden a Python/FastAPI y van en el orden óptimo para la caché.
Comentarios	**40**	✅ Cada instrucción tiene un comentario que explica el PORQUÉ (no qué hace): orden de capas, caché, seguridad, tamaño de imagen. ENTRYPOINT y CMD explican la separación ejecutable/argumentos.
Datos sensibles	**20**	✅ Se identifican los datos sensibles (DB credentials, API keys) y se explica cómo llegan via variables de entorno y AWS Secrets Manager, nunca dentro de la imagen.